

AquaGuard (AquaShield) Capstone Docs — Grading & Review

Overall Grade: D+ (Dangerously Inaccurate)

Summary: Your friend has hallucinated a massive amount of "buzzword" technology that does not exist in your actual codebase. If you submit these documents or present this to the defense board, **you will fail the defense** because the professors will ask to see the Telegram Bot, the Raspberry Pi, and the MongoDB database — none of which exist.

The documents need a major rewrite to reflect the *actual, much smarter* engineering work you did (like the 3.3V pH fix, the FastAPI backend, Firebase, and LOYO Cross-Validation).

Here is the exact breakdown of what is wrong, what to remove, and what to add.

1. Reviewing: AquaShield_Capstone_Project_Report.md

✗ What is completely WRONG (Remove or Replace these):

1. **The Microcontroller:** It says "NodeMCU ESP8266". **Wrong.** You used an **ESP32 (WROOM-32)**.
2. **The Sensors:** It mentions an "Optical Dissolved Oxygen (DO) probe". **Wrong.** You do not have a DO probe. You have a **Thermistor (NTC)** for temperature and a **Servo Motor** for a fish feeder.
3. **The Tech Stack (Massive Hallucination):** The report claims you used "MQTT (Mosquitto)", "Raspberry Pi 4", "Node.js & Express.js", "MongoDB Atlas", and a "Telegram Alert Bot". **Wrong.** You did not use any of these. You used **Firebase Realtime Database** for the cloud, and **Python (FastAPI / Uvicorn)** for the ML backend.
4. **Machine Learning Model 1 Winner:** It claims Random Forest won. **Wrong. Linear Regression** won. Tree-based models (RF/XGBoost) fail at flood prediction because they cannot extrapolate beyond their training data when a record-breaking flood hits. Linear Regression successfully extrapolates the extreme peaks.
5. **Machine Learning Model 2 Purpose:** It claims Model 2 infers "Ammonia and DO". **Wrong.** Model 2 specifically predicts **Pond TDS 60-minutes into the future**.
6. **Cross-Validation:** It mentions "K-Fold Cross Validation". **Wrong.** You used **LOYO (Leave-One-Year-Out) Cross-Validation** specifically to prevent time-series data leakage.

✓ What you need to ADD:

1. **The 3.3V pH Sensor Engineering:** Add a section explaining that you powered the PH4502C sensor at 3.3V (instead of the standard 5V) to protect the ESP32 ADC pins from burning out, and built a custom software calibration tool using Vinegar (pH 2.4) and Baking Soda (pH 8.3). *Professors love real hardware troubleshooting.*
 2. **The ML Safety Buffer:** Add that the ML models actually smoothed out flood peaks and missed some flood days. Explain that you fixed this by setting the UI warning threshold 0.3m–0.5m *below* the 19.05m Danger Level. *This shows extreme engineering maturity.*
-

2. Reviewing: AquaShield_Capstone_Project_Presentation.md

Your presentation slides contain the same fatal hallucinations. Here is the slide-by-slide fix:

Slide 6 & 8: Hardware & Architecture

- **Remove:** "NodeMCU ESP8266", "DO Probe", "Raspberry Pi 4", "MQTT", "Node.js", "MongoDB".
- **Add:** "ESP32", "NTC Thermistor", "Firebase Realtime Database SDK", "Python FastAPI (api_combined.py)".
- **Change:** "Aerator Automation" to "Dual-Pump and Servo Feeder Automation via 2-Channel Relay".

Slide 9: Model 1 (Flood Forecasting)

- **Remove:** "Random Forest achieved highest R2".
- **Add:** "Linear Regression deployed. Tree-based models failed to extrapolate extreme flood peaks; Linear Regression successfully scales to unprecedented water levels."
- **Add:** Mention that it forecasts up to **14 days ahead** (not just 7).

Slide 10: Model 2 (TDS Kinetics)

- **Remove:** Everything about "Ammonia and DO proxy modeling".
- **Add:** "Predicts Total Dissolved Solids (TDS) 60-minutes ahead. Found that a Persistence Baseline and Linear Regression heavily outperformed XGBoost, proving that pond chemistry drifts too slowly for complex decision trees."

Slide 11: Model 3 (Anomaly Detection)

- **Remove:** "Deep Autoencoder". You didn't use neural networks.
- **Add:** Emphasize that Isolation Forest detected **67 anomalies** compared to standard statistical Z-scores (55 anomalies) because it catches *multivariate* anomalies (e.g., pH and Temp both being slightly weird at the same time).

Slide 13: Work Breakdown (The most dangerous slide)

- **Mahfuz's Section:** Delete the claims that he configured "Mosquitto MQTT, Node.js, MongoDB, Socket.io, Telegram Bot, FreeRTOS tasks". If they ask him to show the Node.js code, he will have nothing to show. Replace it with: *"Integrated ESP32 with Firebase Realtime DB, designed the vanilla HTML/JS/CSS frontend dashboard, and built the Firebase analytics historical charting."*
- **Hrithik's Section:** Delete the claims about "residual diagnostics". Change to: *"Built the Python FastAPI microservices, implemented LOYO cross-validation, and performed SHAP explainability analysis."*

Slide 16: Demonstration

- **Remove:** Telegram alerts, MQTT latency.
- **Add:** The **"Simulate Sensor Spike"** feature. Mention that you have a button in the UI that injects a fake pH 9.5 / TDS 450 reading into the Isolation Forest model to instantly turn the dashboard RED, proving the AI works in real-time.

Quick Find-and-Replace Cheat Sheet for Your Friend:

Give this list to your friend to ctrl+F and replace in both documents:

- **Find:** ESP8266 → **Replace with:** ESP32
 - **Find:** MQTT or Mosquitto → **Replace with:** Firebase Realtime Database
 - **Find:** Node.js, Express, MongoDB → **Replace with:** Python FastAPI
 - **Find:** Telegram → **Replace with:** Web Dashboard Alert Banners
 - **Find:** Raspberry Pi → **Replace with:** Local Python Server
 - **Find:** Dissolved Oxygen, DO Probe → **Replace with:** Thermistor (Temperature)
 - **Find:** K-Fold → **Replace with:** Leave-One-Year-Out (LOYO) Cross-Validation
 - **Find:** Model 1: Random Forest → **Replace with:** Model 1: Linear Regression
-

3. Rewritten Presentation Slides (Accurate to the Real Project)

Give this to your friend. This is the exact slide-by-slide content they should use for the presentation, maintaining their original layout but using the **real** project data, findings, and architecture.

Slide 1

AquaGuard: Intelligent Aquaculture Monitoring and Adaptive Protection System

A Full-Stack IoT and Machine Learning Approach to Safeguard Fish Farming in Bangladesh Candidates: Srabon (056) | Mahfuz (057) | Hrithik (071) Supervisor: Sultana Rokeya Naher, Associate Professor

Department of Computer Science & Engineering

Slide 2

Introduction

Fish farming in Bangladesh is highly vulnerable to two unpredictable threats: sudden monsoonal river flooding and rapid deterioration of pond water quality. AquaGuard is a dual-horizon protection system combining a physical IoT Edge Device with Cloud-based Machine Learning. It bridges upstream hydrometeorological signals (river forecasts) with localized pond actuation (pumps/feeders). Empowers farmers with actionable, bilingual (Bengali/English) alerts via a real-time web dashboard.

Department of Computer Science & Engineering

Slide 3

Problem Statement

Macro Threat (Floods): Sudden river swells can wash away an entire season's fish stock in hours. Existing national warnings are often too broad and lack specific water-level metrics for individual farming zones. Micro Threat (Water Quality): Unseen spikes in toxicity (e.g., pH imbalances or sudden TDS surges from fertilizer runoff) cause mass fish mortality before farmers visually notice a problem. Complexity Barrier: Rural farmers

lack access to localized, easy-to-understand predictive data that tells them exactly what physical actions to take.

Department of Computer Science & Engineering

Slide 4

Project Objectives

1. Hardware: Develop a low-cost, ESP32-based IoT edge node to continuously monitor pond pH, TDS, Temperature, and Water Level.
2. Actuation: Automate localized pond protection through 2-channel relay-controlled water pumps and a servo motor feeder.
3. Prediction: Train three specialized Machine Learning models for 14-day river flood forecasting, 60-minute TDS projection, and real-time anomaly detection.
4. Accessibility: Deliver a lightweight, zero-build web dashboard running on free cloud infrastructure (Firebase) with actionable farmer advice.

Department of Computer Science & Engineering

Slide 5

Methodology

Data Collection: Gathered historical water levels from BWDB (Jamuna River) and ERA5 catchment precipitation; collected 2023 hourly IoT sensor logs for pond modeling. Hardware Assembly: Engineered an ESP32 circuit with specific voltage-step-down protections for safe analog-to-digital (ADC) sensor readings. ML Pipeline: Evaluated 10+ regression models using strict Leave-One-Year-Out (LOYO) cross-validation to prevent time-series data leakage. Cloud Integration: Unified the system using Firebase Realtime Database for telemetry and a Python FastAPI backend for AI microservices.

Department of Computer Science & Engineering

Slide 6

Hardware Architecture (IoT Edge Node)

Microcontroller: ESP32 (WROOM-32) with integrated Wi-Fi. Sensors: - pH Probe (PH4502C) powered safely at 3.3V logic. - Ultrasonic Rangefinder (HC-SR04) with a 1k Ω /2k Ω voltage divider on the ECHO pin to protect the ESP32. - Thermistor (NTC) with a 4.7k Ω pull-down resistor for temperature. - TDS Probe (3.3V logic). Actuators: 2-Channel Active-LOW Relay module controlling 2 external submersible pumps, plus 1 Servo motor (feeder).

Department of Computer Science & Engineering

Slide 7

Software & Cloud Architecture

Firmware: C++ (Arduino IDE) utilizing the Firebase ESP32 Client SDK for direct cloud synchronization every 5 minutes. Cloud Database: Firebase Realtime Database (RTDB) providing sub-second latency for live sensor telemetry and pump command states. Web Dashboard: Vanilla HTML, CSS, and JavaScript. No heavy frameworks, ensuring fast loading on rural 3G connections. Includes a custom pH Calibration Wizard. ML Backend: Python FastAPI (uvicorn) REST API hosting all three pickled AI models locally on Port 8000.

Department of Computer Science & Engineering

Slide 8

System Architecture Diagram

[IoT Edge Device (ESP32)] | (Wi-Fi) v [Firebase Realtime Database] <---> [Web Dashboard (Farmer UI)] | (REST API) v [Python FastAPI Backend] / | Model 1 (Flood) Model 2 (TDS) Model 3 (Anomaly)

Department of Computer Science & Engineering

Slide 9

Model 1: River Water-Level Forecasting

Target: Predict Jamuna River (Bahadurabad) water levels 1 to 14 days ahead. Winner: Linear Regression (RMSE: 0.075m, R^2 : 0.998). Key Scientific Finding: Tree-based ensembles (XGBoost/Random Forest) failed to extrapolate beyond their training maximums. During unprecedented record floods, they flatten out. Linear Regression successfully scales to extreme peaks. Safety Buffer Implementation: ML models inherently smooth out peaks, missing some flood days. We mitigated this by setting the UI alert to trigger 0.5 meters BELOW the official 19.05m Danger Level.

Department of Computer Science & Engineering

Slide 10

Model 2: Pond Water Quality (TDS) Kinetics

Target: Predict Total Dissolved Solids (TDS) 60-minutes into the future to preempt toxic runoff spikes. Features: 6-hour rolling means and t-1 to t-12 historical lag windows. Winner: Persistence Baseline (R^2 : 0.942) and Linear Regression (R^2 : 0.888) vastly outperformed XGBoost (R^2 : 0.711). Conclusion: Pond chemistry drifts very slowly over a 1-hour window. Complex decision trees over-complicated the math and overfit to minor sensor noise. Simpler models proved highly superior.

Department of Computer Science & Engineering

Slide 11

Model 3: Unsupervised Anomaly Detection

Target: Detect dangerous, unseen pond conditions without relying on labeled failure datasets. Algorithm: Isolation Forest (Unsupervised Learning). Features: Absolute sensor values plus rate-of-change deltas (ΔpH , ΔTDS , ΔTemp). Result: Successfully detected 67 anomalies (compared to 55 by standard statistical Z-scores). Advantage: Isolation Forests catch *multivariate* anomalies (e.g., pH is 8.2 and Temp is 32°C — neither is an extreme danger alone, but combined they trigger toxic ammonia spikes).

Department of Computer Science & Engineering

Slide 12

Key Engineering Achievements

3.3V pH Sensor Protection: Standard tutorials power the PH4502C at 5V, which burns out ESP32 ADC pins. We engineered it at 3.3V and wrote a custom 2-point software calibration tool using household Vinegar and Baking Soda. Persistent Flash Calibration: Calibration variables are saved directly to the ESP32's non-volatile storage (NVS) via the web dashboard, requiring zero hardware re-flashing. Bilingual Actionable UI: AI predictions are translated into concrete Bengali commands (e.g., "Raise net heights," "Harvest marketable fish").

Department of Computer Science & Engineering

Slide 13

Work Breakdown

Srabon: Hardware Architecture & Prototyping Engineered the ESP32 sensor node, implemented safe voltage dividers for the ADC pins, calibrated the pH/TDS probes, and integrated the 2-channel pump relays. Mahfuz: Cloud Architecture & Web Dashboard Configured Firebase Realtime DB, designed the responsive UI, built the historical analytics charts, and implemented the remote pump actuation toggles. Hrithik: Machine Learning & Backend Engineering Built the Python FastAPI microservices, trained all 3 predictive models, implemented LOYO cross-validation, and conducted feature importance analysis.

Department of Computer Science & Engineering

Slide 14

Conclusion

Comprehensive Protection: AquaGuard successfully bridges localized IoT automation with macro-level flood forecasting to protect vulnerable aquaculture. Methodological Rigor: Proved that simple, regularized Linear Regression outperforms complex AI models (XGBoost) during unprecedented, record-breaking flood events. Robust Engineering: Delivered a zero-data-loss Firebase cloud pipeline and a hardware prototype specifically engineered to prevent microcontroller burnout. Ready for Scale: The modular architecture allows new rivers (e.g., Sylhet/Surma basin) to be added simply by swapping the input CSV dataset.

Department of Computer Science & Engineering

Slide 15

Future Work

1. Multi-Basin Expansion: Expand Model 1 forecasting to the Surma-Kushiyara basin (Sylhet) to handle rapid 24-hour flash floods alongside slow-rising monsoon floods.
2. Automated Alert Engine: Integrate Twilio or WhatsApp Business APIs to push critical anomaly warnings directly to farmers' phones when they are offline.
3. Closed-Loop Pump Automation: Write physical firmware logic to automatically trigger the ESP32 relays the moment Model 3 detects a severe anomaly, without requiring human clicks.
4. Physical Deployment: Flash the finalized V2 firmware to the hardware prototype and deploy it in an active commercial fish pond for longitudinal field testing.

Department of Computer Science & Engineering

Slide 16

Demonstration

Step 1: Multi-Sensor Live Stream: Show the dashboard pulling live pH, TDS, Temp, and Water Level readings directly from Firebase. Step 2: Flood Forecasting (Model 1): Demonstrate the interactive 14-day hydrograph and bilingual farmer advice cards for the Jamuna River. Step 3: Remote Actuation: Toggle Pump 1 and Pump 2 from the dashboard and watch the Firebase state change instantly. Step 4: Live AI Anomaly Injection: Click the 'Simulate Sensor Spike' button to inject a severe reading (pH 9.5) into the FastAPI backend, turning the UI card RED to prove the Isolation Forest runs in real-time.

Department of Computer Science & Engineering

Slide 17

References

[1] Bangladesh Water Development Board (BWDB), 'Hydrometric Data Portal: Daily Water Levels for Station SW46.9L'. [2] Copernicus Climate Change Service, 'ERA5-Land hourly data from 1950 to present', ECMWF. [3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, 'Isolation Forest,' in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008. [4] Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' JMLR 12, pp. 2825-2830, 2011. [5] S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in Advances in Neural Information Processing Systems (NeurIPS 30), 2017.

Department of Computer Science & Engineering

Slide 18

Q & A

Thank You! Question & Answer Session

We welcome questions, suggestions, and valuable feedback from the respected Examination Committee.

AquaGuard: Intelligent Aquaculture Monitoring and Adaptive Protection System Candidates: Srabon (056) | Mahfuz (057) | Hrithik (071) Supervisor: Sultana Rokeya Naher, Associate Professor

Department of Computer Science & Engineering